

Sprawozdanie

Algorytmy metaheurystyczne i sztuczna inteligencja

Przegląd pięciu zagadnień: problem plecakowy, optymalizacja obciążenia procesorów, problem komiwojażera, optymalizacja rojem cząstek oraz samoorganizująca się sieć Kohonena

Zagadnienie 1

Problem plecakowy

Algorytm genetyczny (podejście 0/1 knapsack)

Opis zagadnienia

Problem plecakowy (0/1 knapsack) polega na wyborze podzbioru przedmiotów, z których każdy ma przypisaną wagę i wartość, tak aby zmaksymalizować łączną wartość zabranych przedmiotów przy jednoczesnym nieprzekroczeniu limitu udźwigu plecaka. W tym rozwiązaniu wygenerowano losowo 20 przedmiotów (wagi i wartości z zakresu 0–100) oraz ustalono limit całkowitej wagi na 200. Problem rozwiązano za pomocą algorytmu genetycznego: każdy osobnik (genom) to wektor wartości logicznych określających, czy dany przedmiot jest zabrany. Funkcja dopasowania sumuje wartość przedmiotów w plecaku i zeruje wynik, jeśli przekroczono limit wagi. W kolejnych pokoleniach wybierani są najlepsi osobnicy (20% najlepszych wyników), z których poprzez krzyżowanie jednopunktowe i mutację losową tworzona jest nowa populacja.

Wnioski

Zastosowanie prostego algorytmu genetycznego pozwala skutecznie eksplorować przestrzeń rozwiązań problemu plecakowego bez konieczności przeglądania wszystkich 2^{20} możliwych kombinacji. Kluczową rolę odgrywa funkcja kary (zerowanie wartości po przekroczeniu limitu wagi), która eliminuje niedopuszczalne rozwiązania z puli reprodukcyjnej. Po 20 pokoleniach populacja stabilizuje się wokół zestawu przedmiotów o wysokiej wartości i akceptowalnej łącznej wadze, co widać w rosnącym "wskaźniku wybieralności" przedmiotów o najkorzystniejszym stosunku wartości do wagi. Mankamentem podejścia jest brak mechanizmu elitaryzmu (najlepszy osobnik nie jest gwarantowany do przetrwania), co przy małej liczbie pokoleń może powodować niewielkie wahania jakości wyniku.

Kod źródłowy

```
import random
import pandas as pd

items_list = []
total_weight_limit = 200

for _ in range(20):
    items_list.append((random.randrange(0, 100), random.randrange(0, 100)))
# print(items_list)
items = items_list
items = pd.DataFrame(items, columns=['weight', 'value'])
items.index.name = 'items'

items.T

def create_population(population_size: int):
    # We create a list of genomes. Population size is the number of outer loop iterations.
    return [[random.randint(0, 5) == 0 for _ in range(len(items))]
            for _ in range(population_size)]

def fitness(genes: list):
    total_weight = 0
    total_value = 0
    for idx, gene in enumerate(genes):
        if gene:
            total_weight += items['weight'][idx]
            total_value += items['value'][idx]
    if total_weight > total_weight_limit:
        return 0
    else:
        return total_value

def select_winners(population: list):
    population_values = []
    for genes in population:
        value = fitness(genes=genes)
        if value > 0:
            population_values.append((value, genes))
    return [genes for value, genes in
            sorted(population_values, key=lambda x: x[0], reverse=True)]

def select_best(winners: list, population: list, percent=0.2):
    limit = int(percent * len(population))
    if len(winners) > limit:
        best_ones = winners[:limit]
```

```

    else:
        best_ones = winners
    return best_ones

def crossover(genes1: list, genes2: list):
    crossover_point = random.randint(1, len(genes1) - 1)
    new_genes = genes1[:crossover_point] + genes2[crossover_point:]
    return new_genes

def mutate(genes: list):
    new_genes = list(genes)
    idx = random.randint(0, len(genes) - 1)
    new_genes[idx] = not bool(genes[idx])
    return new_genes

def random_mutate(genes: list, probability: int = 100):
    if random.randint(0, probability) == 0:
        return mutate(genes)
    else:
        return list(genes)

def next_generation(population: list):
    new_population = []
    all_winners = select_winners(population)

    if len(all_winners) > 0:
        best_ones = select_best(all_winners, population, 0.2)
        for _ in range(len(population)):
            new_genes = crossover(random.choice(best_ones), random.choice(best_ones))
            new_population.append(random_mutate(new_genes))
    else:
        new_population = create_population(len(population))

    return new_population

population = create_population(1000)
for i in range(20):
    new_population = next_generation(population)
    population = new_population

results = pd.concat([pd.DataFrame(population).mean() * 100, items], axis=1)
results.columns = ['selection_rate'] + list(items.columns)
results.index.name = items.index.name
results.T

results['selection_rate'].plot.bar()

```

Optymalizacja obciążenia procesorów

Algorytm wspinaczkowy - przydział zadań do rdzeni

Opis zagadnienia

Zagadnienie dotyczy przydziału 100 zadań (o losowym czasie wykonania z zakresu 10–90) do jednego z 4 procesorów o różnej wydajności (współczynniki szybkości: 1.0, 1.25, 1.5, 1.75). Celem jest takie przypisanie zadań, aby zminimalizować maksymalny czas zakończenia pracy wśród wszystkich procesorów (klasyczny problem szeregowania zadań na maszynach równoległych, ang. makespan minimization). Rozwiązanie wykorzystuje prosty algorytm wspinaczki górskiej (hill climbing): startowy losowy przydział zadań jest iteracyjnie modyfikowany poprzez losową zmianę procesora dla jednego zadania; zmiana jest akceptowana tylko wtedy, gdy nie pogarsza (lub poprawia) wynik. Cały proces powtarzany jest w 10 niezależnych rundach po 100 000 iteracji każda, a wybierany jest najlepszy wynik ze wszystkich rund.

Wnioski

Algorytm wspinaczkowy, mimo swojej prostoty, radzi sobie dobrze z tym zagadnieniem — kolejne rundy dają wyniki bardzo zbliżone do siebie (rozstęp rzędu 2 jednostek czasu wokół wartości ok. 918–920), co sugeruje, że algorytm konsekwentnie trafia w okolice optimum lub bliskiego mu minimum lokalnego. Ponieważ akceptowane są również zmiany "równie dobre" (operator $\leq$), algorytm ma pewną swobodę przemieszczania się po platformach o tym samym koszcie, co pomaga uniknąć utknięcia w bardzo płytkim minimum lokalnym. Wadą tego podejścia jest brak mechanizmu "schładzania" czy akceptacji chwilowo gorszych rozwiązań (jak w symulowanym wyżarzaniu), co teoretycznie ogranicza zdolność do ucieczki z głębszych minimów lokalnych - w praktyce uśrednienie po 10 niezależnych startach częściowo rekompensuje ten brak.

Kod źródłowy

```
import random

N = 100
TASKS = [round(random.uniform(10, 90), 2) for _ in range(N)]
CPU_SPEEDS = [1.0, 1.25, 1.5, 1.75]
NUM_CPUS = 4

def calculate_time(genome: list):
    cpu_times = [0.0] * NUM_CPUS

    for task_idx, cpu in enumerate(genome):
        cpu_times[cpu] += TASKS[task_idx] / CPU_SPEEDS[cpu]

    return max(cpu_times)

def mutate(genome: list):
    new_genome = list(genome)
    task_idx = random.randint(0, N - 1)

    old_cpu = new_genome[task_idx]
    new_cpu = random.randint(0, NUM_CPUS - 1)

    while new_cpu == old_cpu:
        new_cpu = random.randint(0, NUM_CPUS - 1)

    new_genome[task_idx] = new_cpu
    return new_genome

def evolve(iteration_count=100000):
    parent = [random.randint(0, NUM_CPUS - 1) for _ in range(N)]
    parent_cost = calculate_time(parent)

    for _ in range(iteration_count):
        child = mutate(parent)
        child_cost = calculate_time(child)

        if child_cost <= parent_cost:
            parent = child
            parent_cost = child_cost

    return parent_cost, parent

best_global_cost = float('inf')
best_global_genome = []
```

```
print("Start: CPU Optimization\n")
for attempt in range(1, 11):
    cost, genome = evolve(100000)
    print(f"Round {attempt}/10: Shortest time found = {round(cost, 2)}")

    if cost < best_global_cost:
        best_global_cost = cost
        best_global_genome = genome
print("\n" + "=" * 40)
print(f"Final optimal execution time = {round(best_global_cost, 2)}")
```

Problem komiwojażera (TSP)

Algorytm genetyczny z krzyżowaniem porządkowym (OX) i elitaryzmem

Opis zagadnienia

Problem komiwojażera polega na znalezieniu najkrótszej trasy odwiedzającej każde z n miast dokładnie raz i powracającej do miasta startowego. W rozwiązaniu wygenerowano macierz odległości 100×100 (losowe wartości 10–90) i zastosowano algorytm genetyczny operujący na permutacjach miast. Populacja 1000 osobników jest sortowana według kosztu trasy; 5% najlepszych osobników przechodzi do kolejnego pokolenia bez zmian (elitaryzm), a nowe osobniki powstają poprzez krzyżowanie porządkowe (order crossover, OX) par rodziców wybranych spośród 50% najlepszych. Zastosowano dodatkowo mutację inwersji fragmentu trasy oraz mutację zamiany dwóch miast. Mechanizm adaptacyjny zwiększa prawdopodobieństwo obu mutacji (z 10%/2% do 40%/30%), gdy przez 70 kolejnych pokoleń nie następuje poprawa najlepszego wyniku, co ma pomóc w wyjściu ze stagnacji. Cały proces powtórzono w 5 niezależnych uruchomieniach po 2000 pokoleń.

Wnioski

Krzyżowanie porządkowe OX w połączeniu z elitaryzmem pozwoliło obniżyć koszt trasy z ok. 4100–4350 w pierwszym pokoleniu do przedziału 1576–1748 po 2000 pokoleniach, w zależności od uruchomienia. Widoczna jest spora zmienność wyników między niezależnymi uruchomieniami (najlepszy 1575,91, najgorszy 1747,74), co wskazuje, że algorytm bywa podatny na utykanie w różnych minimach lokalnych — stąd zasadność wykonywania kilku niezależnych uruchomień i wyboru najlepszego z nich. Mechanizm adaptacyjnego zwiększania siły mutacji po okresie stagnacji (limit cierpliwości = 70 pokoleń) jest próbą przeciwdziałania przedwczesnej zbieżności populacji, choć na podstawie samych wyników trudno jednoznacznie ocenić jego skuteczność bez porównania z wariantem bez tego mechanizmu. Elitaryzm na poziomie 5% skutecznie zabezpiecza przed utratą najlepszego dotąd znalezionej rozwiązania.

Kod źródłowy

```
import random

size = 100
matrix = [[0 for _ in range(size)] for _ in range(size)]

for i in range(size):
    for j in range(size):
        matrix[i][j] = round(random.uniform(10, 90), 2)

def create_population(population_size: int):
    population = []
    for _ in range(population_size):
        genome_to_shuffle = [i for i in range(0, size)]
        random.shuffle(genome_to_shuffle)
        population.append(genome_to_shuffle)
    return population

def fitness(genes: list):
    total_distance = 0
    for i in range(len(genes) - 1):
        total_distance += matrix[genes[i]][genes[i + 1]]
    total_distance += matrix[genes[-1]][genes[0]]
    return total_distance

def swap_mutation(genes: list):
    new_genes = list(genes)
    idx1, idx2 = random.sample(range(len(genes)), 2)
    new_genes[idx1], new_genes[idx2] = new_genes[idx2], new_genes[idx1]
    return new_genes

def inversion_mutation(genome: list):
    size = len(genome)
    start, end = sorted(random.sample(range(size), 2))
    reversed_fragment = genome[start:end + 1][::-1]
    genome[start:end + 1] = reversed_fragment
    return genome
```

```

def crossover_ox(parent1: list, parent2: list, inversion_chance, swap_chance):
    size = len(parent1)
    child = [-1] * size

    start, end = sorted(random.sample(range(size), 2))
    child[start:end + 1] = parent1[start:end + 1]

    fragment_set = set(child[start:end + 1])
    genes_from_parent2 = [gene for gene in parent2 if gene not in fragment_set]

    pointer = 0
    for i in range(size):
        if child[i] == -1:
            child[i] = genes_from_parent2[pointer]
            pointer += 1

    if random.random() < inversion_chance:
        child = inversion_mutation(child)

    if random.random() < swap_chance:
        child = swap_mutation(child)

    return child

def next_generation(population: list, inversion_chance, swap_chance):
    target_size = len(population)

    costs = [(fitness(genes), genes) for genes in population]
    costs.sort(key=lambda x: x[0])
    best_result = costs[0][0]
    best_genome = costs[0][1]

    sorted_population = [genes for _, genes in costs]

    elite = sorted_population[:int(0.05 * target_size)]
    breeding_pool = sorted_population[:int(0.50 * target_size)]

    new_population = list(elite)

    while len(new_population) < target_size:
        lucky_parent1, lucky_parent2 = random.sample(breeding_pool, 2)
        new_child = crossover_ox(lucky_parent1, lucky_parent2, inversion_chance, swap_chance)
        new_population.append(new_child)

    return new_population[:target_size], best_result, best_genome

global_best_cost = float('inf')
global_best_genome = []

num_runs = 5
num_generations = 2000

for run in range(num_runs):
    print(f"\n" + "=" * 40)
    print(f"START {run + 1} / {num_runs}")
    print("=" * 40)

    population = create_population(1000)
    round_best_cost = float('inf')
    round_best_genome = []

    generations_without_improvement = 0
    patience_limit = 70

    for i in range(num_generations):
        inversion_chance = 0.10
        swap_chance = 0.02

        if generations_without_improvement >= patience_limit:
            inversion_chance = 0.40
            swap_chance = 0.30
            generations_without_improvement = 0

        population, cost, best_route = next_generation(population, inversion_chance, swap_chance)

        if cost < round_best_cost:
            round_best_cost = cost
            round_best_genome = best_route
            generations_without_improvement = 0
        else:
            generations_without_improvement += 1

        if i % 500 == 0:
            print(f"Run {run + 1} | Generation {i}: Cost = {round(cost, 2)}")

    print(f"\n End of run {run + 1}. Best result: {round(round_best_cost, 2)}")

    if round_best_cost < global_best_cost:
        global_best_cost = round_best_cost
        global_best_genome = round_best_genome
        print("NEW GLOBAL RECORD")

```

```
print("\n" + "*" * 40)
print("=== FINAL RESULTS ===")
print("*" * 40)
print(f"Best route found: {global_best_genome}")
print(f"Final minimum distance: {round(global_best_cost, 2)}")
```


Optymalizacja rojem cząstek (PSO)

Particle Swarm Optimization — poszukiwanie maksimum funkcji 2D

Opis zagadnienia

Optymalizacja rojem cząstek (Particle Swarm Optimization) to metaheurystyka inspirowana zachowaniem stad zwierząt, w której populacja "cząstek" porusza się po przestrzeni rozwiązań, kierując się własnym najlepszym dotąd wynikiem (best personal) oraz najlepszym wynikiem całego roju (best global). W tym rozwiązaniu 100 cząstek porusza się w dwuwymiarowej przestrzeni $[0, 2\pi] \times [0, 2\pi]$, a maksymalizowana funkcja celu to suma modułów sum funkcji trygonometrycznych o rosnącej częstotliwości ($\sin(x) + \sin(2x) + \sin(4x) + \sin(8x)$ oraz analogicznie dla $\cos(y)$) — funkcja o wielu lokalnych maksimach, co czyni ją dobrym testem dla algorytmów optymalizacji globalnej. Prędkość każdej cząstki aktualizowana jest na podstawie wag bezwładności ($W=0.7$), składnika kognitywnego ($C1=1.5$, przyciąganie do własnego najlepszego wyniku) oraz składnika społecznego ($C2=1.5$, przyciąganie do najlepszego wyniku roju), z ograniczeniem maksymalnej prędkości do bardzo małej wartości (0.0001). Algorytm wykonano przez 100 000 epok.

Wnioski

Rój zbiegł do maksimum lokalnego o wartości ok. 6,30571 już po ok. 6000 epok (przy prawdziwym maksimum funkcji wynoszącym ok. 6,4943) i pozostał w tym punkcie przez pozostałe 94 000 epok. Nie jest to skutek złego doboru limitu prędkości (max_vel), lecz strukturalna cecha klasycznego PSO: gdy cząstki zbliżają się do najlepszego dotąd znalezionej punktu, różnice (pb-x) oraz (gb-x) w składnikach kognitywnym i społecznym dążą do zera, a jedynym pozostałym czynnikiem sterującym prędkością staje się jej wcześniejsza wartość pomnożona przez wagę inercji $W=0.7$. Ponieważ $W < 1$, prędkość zanika w czasie geometrycznie do zera niezależnie od parametrów — jest to zjawisko znane w literaturze jako stagnacja roju. Gdy cząstki się zatrzymają, algorytm nie dysponuje żadnym mechanizmem (losowym restartem, wstrzykiwaniem różnorodności, mutacją), który pozwoliłby wyjść poza osiągnięty punkt, nawet jeśli nie jest to optimum globalne. W praktyce oznacza to, że poprawa wyniku wymagałaby, dodania mechanizmu przeciwdziałającego stagnacji — np. okresowej reinicjalizacji części populacji lub uruchamiania kilku niezależnych rojów i wyboru najlepszego wyniku, analogicznie do podejścia zastosowanego w problemie komiwojażera i optymalizacji CPU.

Kod źródłowy

```
import random
import math

particle_count = 100
max_vel = 0.03
epoch_count = 50000

W = 0.7 # Inertia
C1 = 1.5 # Cognitive component
C2 = 1.5 # Social component

particle_list = []
global_best_score = -1.0
global_best_cords = (0.0, 0.0)

def calc_score(x, y):
    return abs(math.sin(x) + math.sin(2*x) + math.sin(4*x) + math.sin(8*x)) + \
           abs(math.cos(y) + math.cos(2*y) + math.cos(4*y) + math.cos(8*y))

for _ in range(particle_count):
    rand_x = random.uniform(0, 2 * math.pi)
    rand_y = random.uniform(0, 2 * math.pi)
    rand_v_x = random.uniform(-max_vel, max_vel)
    rand_v_y = random.uniform(-max_vel, max_vel)

    score = calc_score(rand_x, rand_y)

    particle = {
        'cords': (rand_x, rand_y),
        'velocity': (rand_v_x, rand_v_y),
        'pb_cords': (rand_x, rand_y),
        'pb_score': score
    }

    particle_list.append(particle)
```

```

    if score > global_best_score:
        global_best_score = score
        global_best_cords = (rand_x, rand_y)

print(f"Start! Best initial guess: {round(global_best_score, 4)}\n")

for epoch in range(1, epoch_count + 1):

    for particle in particle_list:
        x, y = particle['cords']
        v_x, v_y = particle['velocity']
        pb_x, pb_y = particle['pb_cords']
        gb_x, gb_y = global_best_cords

        r1_x, r1_y = random.random(), random.random()
        r2_x, r2_y = random.random(), random.random()

        new_v_x = (W * v_x) + (C1 * r1_x * (pb_x - x)) + (C2 * r2_x * (gb_x - x))
        new_v_y = (W * v_y) + (C1 * r1_y * (pb_y - y)) + (C2 * r2_y * (gb_y - y))

        new_v_x = max(-max_vel, min(max_vel, new_v_x))
        new_v_y = max(-max_vel, min(max_vel, new_v_y))

        new_x = (x + new_v_x) % (2 * math.pi)
        new_y = (y + new_v_y) % (2 * math.pi)

        particle['velocity'] = (new_v_x, new_v_y)
        particle['cords'] = (new_x, new_y)

        new_score = calc_score(new_x, new_y)

        if new_score > particle['pb_score']:
            particle['pb_score'] = new_score
            particle['pb_cords'] = (new_x, new_y)

            if new_score > global_best_score:
                global_best_score = new_score
                global_best_cords = (new_x, new_y)

    if epoch % 1000 == 0:
        print(f"Epoch {epoch}/{epoch_count} -> Current Global Maximum: {round(global_best_score, 5)}")

print("\n" + "="*50)
print("END")
print(f"Highest maximum found: {round(global_best_score, 5)}")
print(f"Located at coordinates: X = {global_best_cords[0]:.5f}, Y = {global_best_cords[1]:.5f}")
print("="*50)

```

Samoorganizująca się mapa Kohonena (SOM)

Klasyfikacja kształtu świec giełdowych na siatce 10x10

Opis zagadnienia

Samoorganizująca się mapa Kohonena (Self-Organizing Map) to typ nienadzorowanej sieci neuronowej, która odwzorowuje dane wielowymiarowe na siatkę neuronów o niższej wymiarowości, zachowując podobieństwa między danymi wejściowymi (podobne wektory trafiają w pobliższe neurony). W tym rozwiązaniu dane wejściowe pochodzą z rzeczywistych notowań giełdowych akcji Apple (AAPL, świece 2-minutowe z 7 dni, pobrane z Yahoo Finance), a każda świeca przekształcana jest na znormalizowany 3-elementowy wektor opisujący proporcje jej korpusu i cieni (osobno dla świec wzrostowych i spadkowych). Sieć składa się z siatki 10x10 neuronów, z których każdy ma losowo zainicjalizowany, znormalizowany wektor wag 3-elementowy (interpretowany później jako kolor RGB). W trakcie uczenia dla każdego wektora danych wyszukiwany jest neuron zwycięski (najbardziej podobny, na podstawie iloczynu skalarnego), a następnie aktualizowane są wagi tego neuronu oraz jego czterech bezpośrednich sąsiadów (schemat "krzyża"), przy czym sąsiedzi aktualizowani są ze zredukowaną siłą ($h=0.5$) względem zwycięzcy. Trening trwa 50 epok, a wynikiem jest obraz siatki, na którym podobne kolorystycznie obszary odpowiadają neuronom wyuczonym na podobne kształty świec.

Wnioski

Zastosowanie mapy Kohonena do danych giełdowych pozwala w sposób w pełni nienadzorowany pogrupować świece o podobnym kształcie (proporcjach korpusu i cieni) w sąsiadujące ze sobą obszary siatki, co wizualnie przejawia się jako spójne kolorystycznie regiony na wygenerowanej mapie. Ograniczenie sąsiedztwa do najbliższego "krzyża" (4 sąsiadów) zamiast szerszego promienia sąsiedztwa upraszcza obliczenia, lecz przy większej liczbie epok lub mniejszej sieci może prowadzić do wolniejszego "rozprzestrzeniania się" uporządkowania po całej siatce w porównaniu do klasycznego podejścia z promieniem sąsiedztwa malejącym w czasie. Stały (a nie malejący w czasie) współczynnik uczenia ($\eta=0.01$) oznacza, że sieć nie "zamraża się" ostatecznie po zakończeniu treningu — przy dłuższym uczeniu wagi mogłyby wciąż nieznacznie drygać. Warto również zauważyć, że normalizacja wektorów wejściowych do długości jednostkowej i użycie iloczynu skalarnego jako miary podobieństwa jest odpowiednikiem miary kosinusowej — dobrze sprawdza się tu, gdyż liczy się kierunek (proporcje), a nie długość wektora cech świecy.

Kod źródłowy

```
import yfinance as yf
import pandas as pd
import numpy as np
import random
import math
import matplotlib.pyplot as plt

def normalize_vector(a, b, c):
    x = math.sqrt(a ** 2 + b ** 2 + c ** 2)
    if x == 0:
        return (0, 0, 0)
    return (a / x, b / x, c / x)

print("Pobieranie danych z Yahoo Finance")
data = yf.download("AAPL", period="7d", interval="2m")

vectorized_data = []

for open_, high, low, close in data[['Open', 'High', 'Low', 'Close']].itertuples(index=False, name=None):
    if high == low:
        continue

    if open_ > close:
        x1 = (high - open_) / (high - low)
        x2 = (open_ - close) / (high - low)
        x3 = (close - low) / (high - low)
    else:
        x1 = (high - close) / (high - low)
```

```

        x2 = (close - open_) / (high - low)
        x3 = (open_ - low) / (high - low)

        vectorized_data.append(normalize_vector(x1, x2, x3))

print(f"Pobrano i wektoryzowano: {len(vectorized_data)} swiec.")

class KohonenSOM:
    def __init__(self, rows=10, cols=10, eta=0.01, h=0.5):
        self.rows = rows
        self.cols = cols
        self.eta = eta
        self.h = h
        self.grid = self._create_board()

    def _create_board(self):
        array = []
        for i in range(self.rows):
            row = []
            for j in range(self.cols):
                w1, w2, w3 = random.uniform(0, 1), random.uniform(0, 1), random.uniform(0, 1)
                row.append(np.array(normalize_vector(w1, w2, w3)))
            array.append(row)
        return array

    def _get_cross(self, r, c):
        all_neurons = [(r, c)]
        directions = [(-1, 0), (1, 0), (0, -1), (0, 1)]

        for dr, dc in directions:
            nr, nc = r + dr, c + dc
            if 0 <= nr < self.rows and 0 <= nc < self.cols:
                all_neurons.append((nr, nc))
        return all_neurons

    def _update_weights(self, cross_list, current_learned_vector):
        for idx, (neuron_r, neuron_c) in enumerate(cross_list):
            weaken = 1 if idx == 0 else self.h
            old_weight = self.grid[neuron_r][neuron_c]

            new_weight_raw = old_weight + self.eta * weaken * (
                np.array(current_learned_vector) - old_weight)

            a, b, c = new_weight_raw
            self.grid[neuron_r][neuron_c] = np.array(normalize_vector(a, b, c))

    def train(self, data, epochs=50):
        print("Rozpoczeto nauke sieci")
        for epoch in range(epochs):
            for vector in data:
                highest_score = -1
                best_r, best_c = -1, -1

                for r, row in enumerate(self.grid):
                    for c, neuron in enumerate(row):
                        scalar = np.dot(vector, neuron)
                        if highest_score < scalar:
                            highest_score = scalar
                            best_r, best_c = r, c

                if highest_score > -1:
                    cross_list = self._get_cross(best_r, best_c)
                    self._update_weights(cross_list, vector)

            if epoch % 10 == 0 or epoch == epochs - 1:
                print(f"Epoka {epoch}/{epochs} zakonczona.")

        print("Zakonczono uczenie")

    def show_grid(self, title="Mapa Kohonena (RGB = Wagi Swiec)":
        grid_array = np.array(self.grid)
        plt.figure(figsize=(8, 8))
        plt.title(title)
        plt.imshow(grid_array)
        plt.axis('off')
        plt.show()

som_network = KohonenSOM(rows=10, cols=10, eta=0.01, h=0.5)

# Odpalamy trening
som_network.train(vectorized_data, epochs=50)

# Wizualizacja

```

```
som_network.show_grid()
```